

UNIVERSITATEA "ȘTEFAN CEL MARE", SUCEAVA
FACULTATEA DE INGINERIE ELECTRICĂ ȘI ȘTIINȚA CALCULATOARELOR
SPECIALIZAREA CALCULATOARE
AN 2, GRUPA 3123 A

PROIECT DISCIPLINA POO

Aplicație de gestiune a unui spital
(C++)

Autor: Cojocaru Vasilica

Profesor coordonator: dr. Ing. Prodan Remus

Cuprins

- 1.Descrierea proiectului**
- 2.Obiectivele aplicației**
- 3.Concepte OOP utilizate**
- 4.Descrierea claselor**
- 5.Funcționalități implementate**
- 6.Diagrama UML**
- 7.Cerințe implementate**
- 8.Capturi de ecran**
- 9.Concluzii**
- 10. Bibliografie**

TEMĂ PROIECT

1. Descrierea proiectului

Proiectul reprezintă o aplicație de gestiune a unui spital realizată în limbajul C++ utilizând concepte de Programare Orientată pe Obiecte.

Aplicația permite gestionarea pacienților, medicilor, asistenților, programărilor, internărilor și facturilor medicale prin intermediul unui meniu interactiv în consolă.

2. Obiectivele aplicației

Aplicația oferă următoarele funcționalități:

- gestionarea pacienților;
- gestionarea medicilor;
- gestionarea asistenților;
- crearea programărilor medicale;
- internarea pacienților;
- emiterea facturilor;
- filtrarea pacienților după diagnostic;
- afișarea statisticilor spitalului;
- salvarea pacienților într-un fișier text.

3. Concepte OOP utilizate

Încapsulare

Datele sunt stocate în clase și sunt accesate prin intermediul metodelor publice.

Moștenire

A fost utilizată relația:

Angajat → Medic

Angajat → Asistent

Clasele Medic și Asistent reutilizează atributele și metodele clasei de bază Angajat.

Polimorfism

Polimorfismul este realizat prin metoda virtuală:

`calculeazaCostServiciu()`

care este implementată diferit în clasele Medic și Asistent.

Tratarea excepțiilor

Pentru validarea programărilor este utilizată excepția:

`ProgramareInvalidaException`

STL

Pentru gestionarea colecțiilor sunt utilizate containere STL de tip vector:

- vector
- vector
- vector
- vector
- vector
- vector

Factory Pattern

Pentru crearea facturilor este utilizată clasa:

FacturaFactory

Logging

Operațiile importante sunt salvate în fișierul log.txt utilizând clasa Logger.

4. Descrierea claselor

Clasa Pacient

Reține informațiile unui pacient:

- id
- nume
- vârstă
- diagnostic

Metode:

- afisare()
- getId()
- getNume()
- getVarsta()
- getDiagnostic()

Clasa Angajat

Clasa de bază pentru personalul medical.

Conține:

- id
- nume
- salariu

Metodă virtuală:

- calculeazaCostServiciu()

Clasa Medic

Moștenește clasa Angajat.

Conține:

- specializare

Metode:

- afisare()
- calculeazaCostServiciu()

Clasa Asistent

Moștenește clasa Angajat.

Conține:

- secție

Metode:

- afisare()
- calculeazaCostServiciu()

Clasa Programare

Gestionează programările medicale.

Conține:

- pacient
- medic
- data
- ora

Metode:

- afisare()

Clasa Factura

Gestionează emiterea facturilor.

Conține:

- pacient
- medic

- cost

Metode:

- afisare()
- calcul cost servicii

Clasa Internare

Gestionează internările pacienților.

Conține:

- pacient
- data internării
- salon

Metode:

- afisare()

Clasa Logger

Salvează operațiile importante în fișierul log.txt.

Operații logate:

- emiterea facturilor;
- internarea pacienților.

Clasa FacturaFactory

Implementează Factory Pattern pentru crearea obiectelor de tip Factura.

Clasa ProgramareInvalidaException

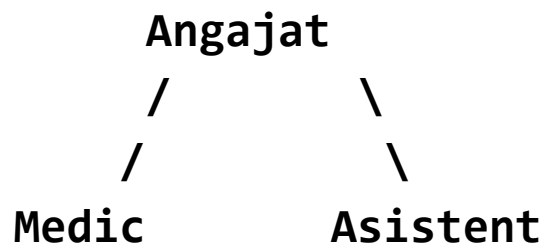
Excepție personalizată utilizată pentru validarea programărilor.

5. Funcționalități implementate

Aplicația permite:

- 1. Afișarea tuturor pacienților**
- 2. Afișarea tuturor medicilor**
- 3. Afișarea tuturor asistenților**
- 4. Afișarea programărilor**
- 5. Afișarea internărilor**
- 6. Afișarea facturilor**
- 7. Adăugarea unui pacient**
- 8. Crearea unei programări**
- 9. Emiterea unei facturi**
- 10. Internarea unui pacient**
- 11. Filtrarea pacienților după diagnostic**
- 12. Afișarea statisticilor spitalului**
- 13. Adăugarea unui medic**
- 14. Adăugarea unui asistent**

6. Diagrama UML



Pacient ----- Programare ----- Medic

Pacient ----- Factura ----- Medic

Pacient ----- Internare

FacturaFactory ----- Factura

Logger ----- Factura

Logger ----- Internare

ProgramareInvalidaException ----- Programare

7. Cerințe implementate

Cerințe obligatorii

- ✓ Clase: Pacient, Medic, Programare, Factura
- ✓ Moștenire: Angajat → Medic, Asistent
- ✓ Polimorfism pentru calculul costurilor
- ✓ Excepții personalizate

- ✓ Logging pentru internări și facturi
- ✓ Teste unitare
- ✓ Organizare pe fișiere .h și .cpp
- ✓ Utilizarea STL
- ✓ Git pentru gestionarea proiectului

Cerințe facultative

- ✓ Pattern Factory pentru crearea facturilor
- ✓ Filtrare pacienți după diagnostic
- ✓ Salvare pacienți în fișier text

Observație: Persistența JSON/XML nu a fost implementată.

8.Capturi de ecran

```
C:\Users\Vasilica\Desktop\Ge: x + v
=====
GESTIUNE SPITAL
=====

DATE PRINCIPALE
[1] Afisare pacienti
[2] Afisare medici
[3] Afisare asistenti

OPERATIUNI SPITAL
[4] Afisare programari
[5] Afisare internari
[6] Afisare facturi

GESTIONARE DATE
[7] Adaugare pacient
[8] Creare programare
[9] Emitere factura
[10] Internare pacient
[11] Filtrare pacienti dupa diagnostic
[12] Statistici spital
[13] Adaugare medic
[14] Adaugare asistent

[0] Iesire

=====
Alege optiunea: |
```

Fig 1. Meniul principal al aplicatiei

```
C:\Users\Vasilica\Desktop\Ge: x + v
=====
GESTIUNE SPITAL
=====

DATE PRINCIPALE
[1] Afisare pacienti
[2] Afisare medici
[3] Afisare asistenti

OPERATIUNI SPITAL
[4] Afisare programari
[5] Afisare internari
[6] Afisare facturi

GESTIONARE DATE
[7] Adaugare pacient
[8] Creare programare
[9] Emitere factura
[10] Internare pacient
[11] Filtrare pacienti dupa diagnostic
[12] Statistici spital
[13] Adaugare medic
[14] Adaugare asistent

[0] Iesire

=====
Alege optiunea: |
```

Fig 2. Afisarea pacientilor existenti in sistem

```
C:\Users\Vasilica\Desktop\Ge: x + v
-----
Pacienti disponibili:
1. Ion Popescu
2. Maria Stan
3. Andrei Ionescu
Alege pacientul: 2

Medici disponibili:
1. ID medic: 1
Nume: Dr. Maria Ionescu
Specializare: Cardiologie
Cost consultatie: 200 lei

2. ID medic: 2
Nume: Dr. Andrei Popescu
Specializare: Neurologie
Cost consultatie: 200 lei

3. ID medic: 3
Nume: Dr. Elena Stan
Specializare: Pediatrie
Cost consultatie: 200 lei

Alege medicul: 1
Data programarii: 04.06.2026
Ora programarii: 13:00

Programare creata cu succes!

Apasa Enter pentru a continua...|
```

Fig 3. Crearea unei programari medicale

```
C:\Users\Vasilica\Desktop\Ge: x + v
=====
GESTIUNE SPITAL
=====

EMITERE FACTURA
-----
Pacienti disponibili:
1. Ion Popescu
2. Maria Stan
3. Andrei Ionescu
Alege pacientul: 1

Medici disponibili:
1. ID medic: 1
Nume: Dr. Maria Ionescu
Specializare: Cardiologie
Cost consultatie: 200 lei

2. ID medic: 2
Nume: Dr. Andrei Popescu
Specializare: Neurologie
Cost consultatie: 200 lei

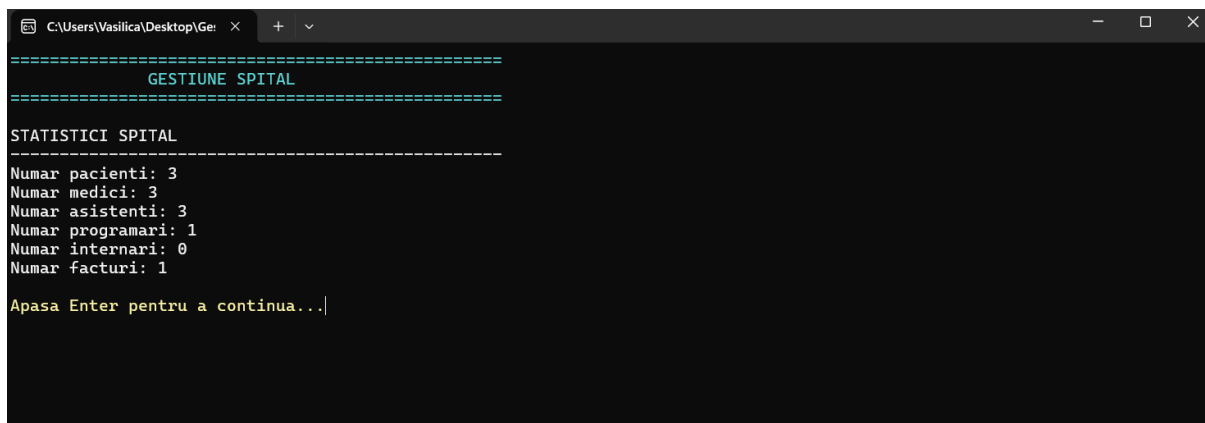
3. ID medic: 3
Nume: Dr. Elena Stan
Specializare: Pediatrie
Cost consultatie: 200 lei

Alege medicul: 3

Factura emisa cu succes!
===== FACTURA =====
ID Factura: 1
Pacient: Ion Popescu
Total de plata: 200 lei

Apasa Enter pentru a continua...|
```

Fig 4. Emiterea unei facturi medicale



```
=====
GESTIUNE SPITAL
=====

STATISTICI SPITAL
-----
Numar pacienti: 3
Numar medici: 3
Numar asistenti: 3
Numar programari: 1
Numar internari: 0
Numar facturi: 1

Apasa Enter pentru a continua...|
```

Fig 5. Statistici generate de aplicatie

9. Concluzii

Proiectul demonstrează utilizarea principalelor concepte ale Programării Orientate pe Obiecte în C++.

Au fost utilizate moștenirea, polimorfismul, excepțiile personalizate, containere STL, Factory Pattern și mecanisme de logging pentru realizarea unei aplicații complete de gestiune a unui spital.

Aplicația poate fi extinsă în viitor prin utilizarea fișierelor JSON/XML sau a unei baze de date.

10. Bibliografie

<https://chatgpt.com/c/6a1f0109-1718-8394-85c3-e90c7faeeb56>